CIFAR-10 Image Classification Using Deep Convolutional Neural Network — Report

1. Methodology

1.1 Model Architecture

The model is a **deep Convolutional Neural Network (CNN)** built using the TensorFlow/Keras. It is designed for multi-class image classification on the CIFAR-10 dataset, which contains **60,000 colour images** (32×32×3) across **10 classes**: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck (50,000 training / 10,000 test).

1.2 Layers Used

The network consists of **three convolutional blocks** followed by **two fully-connected (dense) layers** and an **output layer**:

Layer No.	Layer Type	Filters Units	Kernel/Pool size	Output Shape	Parameters
1	Conv2D	32	3×3	(30, 30, 32)	896
2	BatchNormalization	—	—	(30, 30, 32)	128
3	MaxPooling2D	—	2×2 (stride 2)	(15, 15, 32)	0
4	Conv2D	64	3×3	(13, 13, 64)	18,496
5	BatchNormalization	—	—	(13, 13, 64)	256
6	MaxPooling2D	—	2×2 (stride 2)	(6, 6, 64)	0
7	Conv2D	128	3×3	(4, 4, 128)	73,856
8	BatchNormalization	—	—	(4, 4, 128)	512
9	MaxPooling2D	—	2×2 (stride 2)	(2, 2, 128)	0
10	Flatten	—	—	(512)	0
11	Dense	256	—	(256)	131,328
12	Dropout (0.5)	—	—	(256)	0
13	Dense	128	—	(128)	32,896
14	Dropout (0.3)	—	—	(128)	0
15	Dense (Output)	10	—	(10)	1,290

1.3 Activation Functions, Loss & Optimizer

Component	Choice	Rationale
Hidden activations	`ReLU`	Efficient gradient flow; avoids vanishing-gradient problem
Output activation	`Softmax` (10 units)	Produces a valid probability distribution over 10 classes
Loss function	`Sparse Categorical Cross-Entropy`	Handles integer-encoded labels directly for multi-class tasks
Optimizer	`Adam` (default lr ≈ 0.001)	Adaptive learning rate; fast convergence; widely used baseline

1.4 Workflow

1. **Data Loading:** `tf.keras.datasets.cifar10.load_data()` — 50,000 train + 10,000 test images.
2. **Preprocessing (Training Set):** Pixel rescaling to `[0, 1]`, plus real-time data augmentation via `ImageDataGenerator` (shear 0.2, zoom 0.2, horizontal flip, rotation ±15°, width/height shift 0.1).
3. **Preprocessing (Test Set):** Pixel rescaling to `[0, 1]` only — **no** augmentation (preserves test integrity).
4. **Training:** `cnn.fit()` for **25 epochs**, batch size **64**, with validation on the test set.
5. **Evaluation:** Final test accuracy/loss computed, plus single-image prediction demo.

1.5 Hyperparameters

Hyperparameter	**Value**
Epochs	25
Batch Size	64
Learning Rate	Default Adam (≈ 0.001)
Conv Filters	32 → 64 → 128
Kernel Size	3×3 (all convolutions)
Pool Size / Stride	2×2 / 2
Dense Units	256 → 128 → 10
Dropout Rates	0.5 (first), 0.3 (second)
Augmentation	shear=0.2, zoom=0.2, h-flip, rotation=±15°, w/h-shift=0.1

---

2. Results and Analysis

Three distinct training strategies were evaluated on the **same CNN architecture** with a total budget of **25 epochs** each:

1. **Standard Training** — Full dataset used for all 25 epochs.
2. **Progressive Dataset Revelation** — Training data revealed in 5 stages (10% → 25% → 50% → 75% → 100%), 5 epochs per stage.

3. **Curriculum Learning (Prediction Difficulty)** — A scorer model ranks samples by difficulty; the final model trains on progressively harder subsets (easiest 20% → 40% → 60% → 80% → 100%), 5 epochs per stage.

2.1 Classification Metrics — All Three Strategies

Strategy	Test Accuracy	Test Loss
Standard Training	**75.88 %**	**0.6819**
Progressive Revelation	**74.00 %**	**0.7597**
Curriculum Learning	**71.38 %**	**0.8304**

The **Standard Training** strategy achieves the best performance with ~76% accuracy and the lowest test loss. Progressive Revelation follows closely at ~74%, while Curriculum Learning lags at ~71%.

2.2 Training & Validation Curves — Per Strategy

Standard Training (cifar10_cnn.ipynb):

Epoch	Train Acc	Val Acc	Train Loss	Val Loss
1	0.2953	0.5124	2.0151	1.3735
5	0.5892	0.6000	1.1817	1.1547
10	0.6633	0.7184	0.9924	0.8080
15	0.6998	0.7232	0.8826	0.7997
20	0.7195	0.7148	0.8226	0.8205
25	0.7375	0.7588	0.7883	0.6819

With the standard strategy, the model sees the **full 50,000 training images at every epoch**. Training accuracy rises steadily from ~30% to ~74%, while validation accuracy settles around 71–76% from epoch 10 onward. The small train–validation gap (~2 pp) indicates good generalisation aided by data augmentation, BatchNormalization, and Dropout.

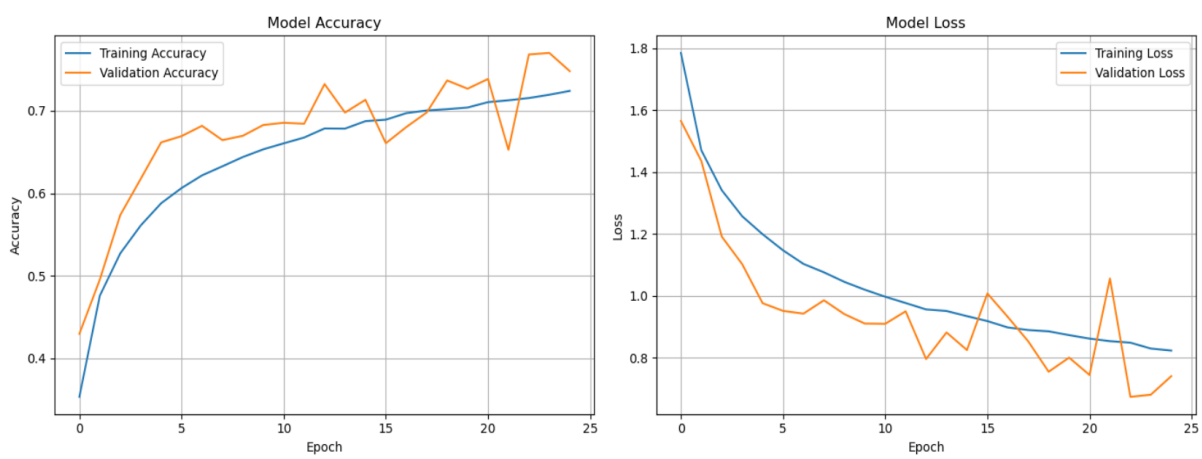

Figure 1: Full Dataset Training

Progressive Dataset Revelation (cifar10_progressive.ipynb):

Stage	Data %	End Train Acc	End Val Acc	End Val Loss
1 (Epochs 1–5)	10% (5,000)	0.3532	0.2857	1.8975
2 (Epochs 6–10)	25% (12,500)	0.5043	0.5535	1.2507
3 (Epochs 11–15)	50% (25,000)	0.5976	0.5912	1.1526
4 (Epochs 16–20)	75% (37,500)	0.6512	0.6777	0.9355
5 (Epochs 21–25)	100% (50,000)	0.6939	0.7400	0.7597

The progressive strategy starts training on a very small subset, leading to initially low and unstable validation accuracy (only ~29% after stage 1). Performance ramps up as more data is introduced, reaching 74% at the final stage. The key drawback is that the model spends its first 5 epochs on only 5,000 images, which limits the diversity of patterns learned early on. By the time the full dataset is available (stage 5), the model has only 5 remaining epochs to leverage the complete training set — fewer than the 25 full epochs the standard strategy gets.

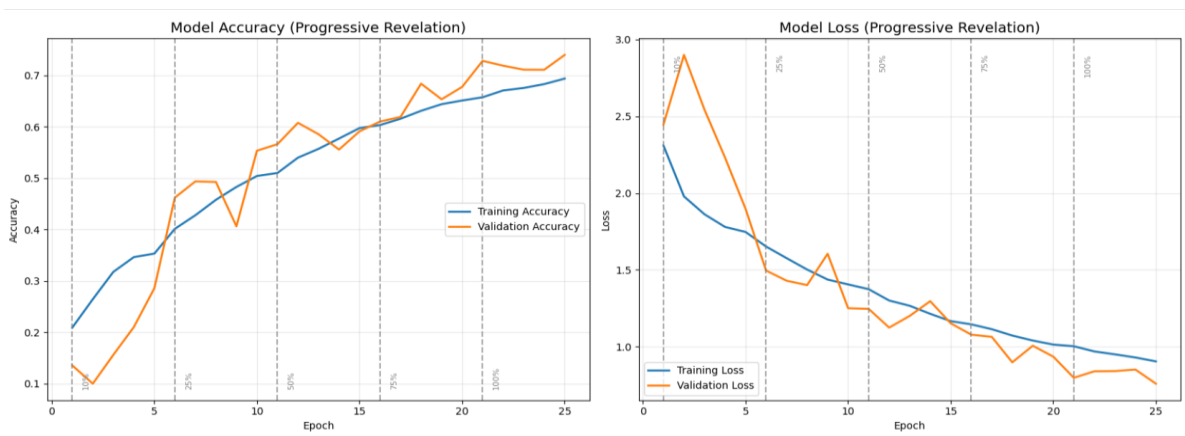

Figure 2: Progressive Dataset Training

Curriculum Learning (cifar10_curriculum.ipynb):

Stage	Data % (by difficulty)	End Train Acc	End Val Acc	End Val Loss
1 (Epochs 1–5)	Easiest 20%	0.6068	0.4984	1.5042
2 (Epochs 6–10)	Easiest 40%	0.5862	0.5989	1.1489
3 (Epochs 11–15)	Easiest 60%	0.6123	0.6440	1.0335
4 (Epochs 16–20)	Easiest 80%	0.6362	0.6624	0.9744
5 (Epochs 21–25)	100%	0.6780	0.7138	0.8304

Curriculum learning ranks data by difficulty and trains on easy samples first, leading to quick early gains (60% training accuracy) but weaker generalisation. As harder samples are added, performance improves, yet final test accuracy (71.38%) is lowest because difficulty scoring is noisy, early training lacks diversity, and harder, more informative examples get less training time.

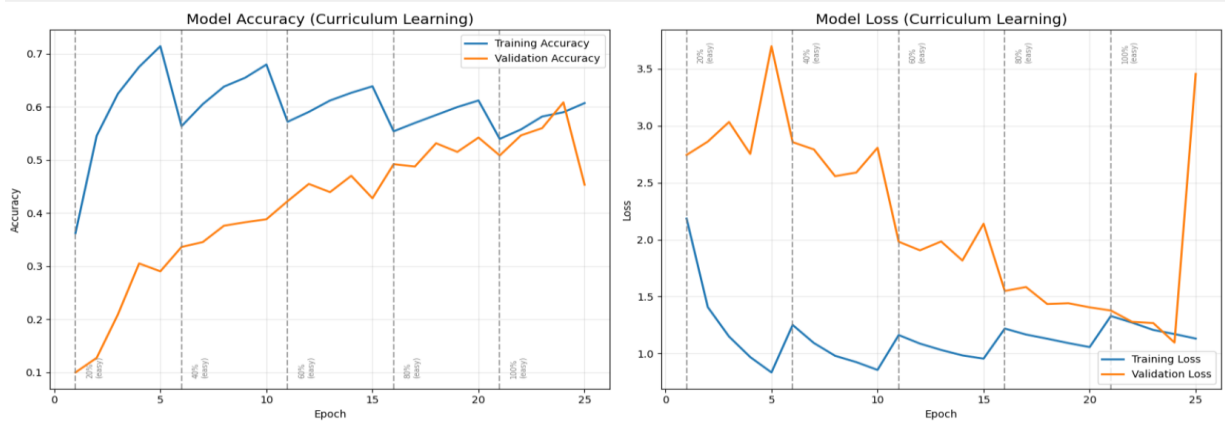

Figure 3: Curriculum Dataset Training

2.3 Comparison of the Three Strategies

Strategy	Test Accuracy	Test Loss	Key Advantage	Key Limitation
Standard Training	**75.88 %**	**0.6819**	Full data exposure every epoch; most stable convergence	No curriculum benefit; treats all samples equally
Progressive Revelation	74.00 %	0.7597	Encourages learning from less to more; moderate regularisation effect	Only 5 epochs on full dataset; slow initial progress
Curriculum Learning	71.38 %	0.8304	Easy-first may stabilise early training	Scorer-induced bias; hard examples under-trained; extra compute cost for scorer

---

3. Conclusion

The **Standard Training strategy was the clear winner**, achieving the highest test accuracy (**75.88%**) and the lowest test loss (**0.6819**). Progressive Revelation came second at 74.00%, while Curriculum Learning trailed at 71.38%. The standard approach benefits from exposing the model to the full diversity of the training set at every epoch.

The **Progressive Revelation** strategy, while conceptually appealing, suffers from the fact that the model spends a significant portion of its training budget on small subsets, limiting early-stage learning. The **Curriculum Learning** strategy performed the worst primarily because the scorer model was itself weak (only 3 epochs of pre-training), leading to noisy difficulty rankings. Additionally, easy-first ordering can reduce sample diversity in early training, and the hardest — often most informative — examples receive the least training time.

Overall, **standard full-data training performs best**. Progressive and curriculum strategies may improve with a better scorer or a mixed (easy + hard) sampling schedule rather than strict ordering.